

Architecture du systeme - Smart Parking (Backend)

1. Vue d'ensemble

Le backend Smart Parking suit une architecture en couches avec Spring Boot:

- **Client**: Frontend web et application mobile via API REST.
- **Securite**: JWT + Spring Security pour authentication et autorisation.
- **Controllers**: points d'entree HTTP (`/api/**`).
- **DTO**: objets Request/Response pour isoler l'API des entites JPA.
- **Services**: logique metier (reservation, paiement, administration).
- **Repositories**: acces aux donnees via Spring Data JPA.
- **Entities**: modele de donnees persiste en MySQL.

Flux principal:

`Client -> Security(JWT) -> Controller -> Service -> Repository -> MySQL`

2. Structure du projet

Backend/

```
■■■■ pom.xml
■■■■ src/main/java/org/example/smartparking/
■ ■■■■ controller/    # Endpoints REST
■ ■■■■ dto/           # Objets d'entree/sortie API
■ ■■■■ entity/        # Entites JPA + enums
■ ■■■■ exception/     # Gestion centralisee des erreurs
■ ■■■■ repository/    # Couche d'accès aux données
■ ■■■■ security/      # JWT + configuration Spring Security
■ ■■■■ service/       # Interfaces metier
■ ■■■■ service/impl/  # Implementations metier
■■■■ src/main/resources/
■ ■■■■ application.properties
■ ■■■■ schema.sql
■■■■ ARCHITECTURE.md
```

3. Modele de donnees (ERD)

Tables metier principales:

- `users`
- `parkings`
- `parking\_slots`
- `reservations`
- `payments`
- `wallets`
- `wallet\_transactions`

Relations essentielles:

- ****User -> Reservation****: 1..N
- ****User -> Payment****: 1..N
- ****Parking -> ParkingSlot****: 1..N
- ****ParkingSlot -> Reservation****: 1..N
- ****Reservation -> Payment****: 1..1
- ****User -> Wallet****: 1..1
- ****Wallet -> WalletTransaction****: 1..N

4. Couche API REST

Authentication (`/api/auth`)

- `POST /api/auth/register` : inscription utilisateur
- `POST /api/auth/login` : connexion utilisateur
- `POST /api/auth/register-admin` : creation compte admin

Parkings (`/api/parkings`)

- `GET /api/parkings` : liste des parkings
- `GET /api/parkings/{id}` : detail parking
- `GET /api/parkings/active` : parkings actifs
- `GET /api/parkings/search?name=...` : recherche
- `POST /api/parkings` : creation (admin)
- `PUT /api/parkings/{id}` : modification (admin)
- `DELETE /api/parkings/{id}` : suppression (admin)

Places (`/api/parking-slots`)

- `GET /api/parking-slots/{id}` : detail place
- `GET /api/parking-slots/parking/{parkingId}` : places d'un parking

- `GET /api/parking-slots/available/{parkingId}` : places disponibles
- `POST /api/parking-slots` : creation (admin)
- `PUT /api/parking-slots/{id}` : modification (admin)
- `DELETE /api/parking-slots/{id}` : suppression (admin)

Reservations (`/api/reservations`)

- `POST /api/reservations` : creer reservation
- `GET /api/reservations/{id}` : detail reservation
- `GET /api/reservations/my-reservations` : reservations du user connecte
- `PUT /api/reservations/{id}/cancel` : annuler reservation

Paielements (`/api/payments`)

- `GET /api/payments/reservation/{id}` : paiement d'une reservation
- `GET /api/payments/my-payments` : paiements du user connecte

Wallet (`/api/wallet`)

- `GET /api/wallet/balance` : solde du portefeuille
- `POST /api/wallet/top-up` : recharge
- `POST /api/wallet/pay` : paiement via wallet
- `GET /api/wallet/transactions` : historique

Administration (`/api/admin`)

- `GET /api/admin/dashboard` : statistiques globales
- `GET /api/admin/users` : liste utilisateurs
- `GET /api/admin/reservations` : liste reservations
- `GET /api/admin/payments` : liste paiements

5. Securite JWT

Parcours d'authentification

1. Le client appelle `POST /api/auth/login` avec email + mot de passe.
2. `AuthenticationManager` valide les identifiants.
3. `JwtTokenProvider` genere un token JWT.
4. Le token est retourne dans `AuthResponse`.
5. Le client envoie `Authorization: Bearer ` sur les routes protegees.
6. `JwtAuthenticationFilter` valide signature + expiration + utilisateur.

7. `SecurityConfig` applique les regles d'accès par rôle.

Composants de sécurité

- `SecurityConfig`
- `JwtTokenProvider`
- `JwtAuthenticationFilter`
- `JwtAuthenticationEntryPoint`
- `CustomUserDetailsService`

Règles d'accès

- **Public**: `/api/auth/`, consultation des parkings/places disponibles
- **Authentifié**: réservations, paiements, wallet
- **Admin**: endpoints `/api/admin/` + écriture/suppression parkings/places

6. Choix techniques

- **Spring Boot + Spring Web** pour API REST
- **Spring Data JPA + Hibernate** pour persistance
- **MySQL** comme base relationnelle
- **JWT stateless** pour authentification scalable
- **Bean Validation** pour valider les payloads
- **@RestControllerAdvice** pour erreurs homogènes

7. Scalabilité et qualité

Bonnes pratiques appliquées:

1. Architecture en couches claire
2. Services interfaces + implémentations
3. DTO pour stabilité du contrat API
4. Réponses standardisées (`ApiResponse`)
5. Transactions (`@Transactional`)
6. Index SQL sur colonnes fréquentes

Recommandations de progression:

1. Pagination sur endpoints de liste
2. Cache Redis/Caffeine pour disponibilité des places
3. Rate limiting (Bucket4j / API Gateway)

4. Observabilite (Actuator, Prometheus, Grafana)
5. Containerisation (Docker) + orchestration (Kubernetes)

8. Variables d'environnement conseilles

Pour eviter les secrets dans le code:

- `DB\_PASSWORD`
- `JWT\_SECRET`
- `AUTH\_SECRET` (frontend)
- `GOOGLE\_AUTH\_CLIENT\_SECRET`
- `GITHUB\_AUTH\_CLIENT\_SECRET`
- `MAPBOX\_ACCESS\_TOKEN`

9. Conclusion

Le backend Smart Parking est organise pour separer proprement la securite, la logique metier et la persistance. Cette structure facilite:

- l'evolution fonctionnelle (nouvelles routes et modules),
- la maintenance (isolation des responsabilites),
- la montee en charge (stateless JWT, optimisation SQL, extension cache/queue).